



UNIVERSIDAD DE
GUANAJUATO



La notación Big O

Instituto Tecnológico Sanmiguelense de Estudios Superiores

“Responsabilidad histórica de ser vanguardia”

Luis Ángel Iván Chávez Zúñiga

“Estructuras”

Andres Alexis Espinosa Ramirez

/30/01/2026/

La notacion Big o

La notación Big O es una herramienta matemática utilizada en ciencias de la computación para describir el comportamiento de los algoritmos en términos de eficiencia temporal y espacial. Su propósito principal es ofrecer una medida abstracta que permita comparar algoritmos sin depender de factores externos como el hardware o el lenguaje de programación.

Características fundamentales:

Cota Superior (upper bound):

Big O establece un límite superior para el tiempo de ejecución de un algoritmo. Esto significa que sin importar las variaciones en la entrada el algoritmo nunca será más lento que la función descrita por la notación. Por ejemplo, si un algoritmo se describe como $O(n^2)$, su tiempo de ejecución no crecerá más rápido que una función cuadrática respecto al tamaño de la entrada.

Comportamiento Asintotico:

La notación se centra en el comportamiento del algoritmo cuando el tamaño de la entrada tiende a infinito. Esto permite ignorar detalles menores y enfocarse en cómo el algoritmo escala en problemas grandes.

Análisis del Peor Escenario (worst case):

Aunque existen otras notaciones como Omega y Theta, Big O se utiliza principalmente para describir el peor caso. Esto garantiza que el análisis sea conservador y útil en contextos donde la eficiencia es crítica.

Independencia del Hardware y Lenguaje:

Big O no depende de la velocidad del procesador, la memoria disponible ni el lenguaje de programación. Es una medida teórica que describe la eficiencia algorítmica en términos universales.

Simplificación de Constantes:

En el análisis asintotico se eliminan constantes y factores menores. Por ejemplo, un algoritmo que tarda $3n+5$ pasos se describe simplemente como $O(n)$, ya que el crecimiento lineal domina sobre los valores constantes.

Dominancia de Términos:

Cuando una función contiene múltiples términos, se conserva únicamente el de mayor crecimiento. Por ejemplo, $n^2+n+100$ se simplifica a $O(n^2)$.

Crecimiento Relativo:

Big O permite comparar algoritmos según la rapidez con que crece su tiempo de ejecución. Un algoritmo $O(n)$ será más eficiente que uno $O(n^2)$ para entradas grandes, aunque en casos pequeños las diferencias puedan ser mínimas.

Tipos comunes de complejidad:**Constante $O(1)$:**

El tiempo de ejecución no depende del tamaño de la entrada. Ejemplo: acceder a un elemento específico en un arreglo.

Logarítmica $O(\log n)$:

Crece lentamente incluso con entradas grandes. Ejemplo: búsqueda binaria en una lista ordenada.

Lineal $O(n)$:

El tiempo de ejecución aumenta proporcionalmente al tamaño de la entrada. Ejemplo: recorrer todos los elementos de un arreglo.

Lineal-logarítmica $O(n \log n)$:

Común en algoritmos de ordenamiento eficientes como Merge Sort y Quick Sort.

Cuadrática $O(n^2)$:

El tiempo de ejecución crece rápidamente, es común en algoritmos de fuerza bruta que comparan pares de elementos.

Exponencial $O(2^n)$:

Crece extremadamente rápido, como en problemas de combinatoria o algoritmos recursivos sin optimización.

Factorial $O(n!)$:

Crece aún más rápido que el exponencial, como en la generación de todas las permutaciones posibles de un conjunto.

La importancia en la ciencia de la computación

La notación Big O es esencial porque permite:

Evaluar la escalabilidad de los algoritmos.

Comparar diferentes enfoques para resolver un mismo problema.

Identificar cuellos de botella en sistemas grandes.

Guiar el diseño de software eficiente y sostenible.

En la práctica, los ingenieros de software utilizan Big O para decidir qué algoritmos son adecuados según el tamaño esperado de los datos. Por ejemplo, un algoritmo $O(n^2)$ puede ser aceptable para conjuntos pequeños, pero ineficiente para millones de elementos.

Referencias:

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Knuth, D. E. (1997). *The art of computer programming, Volume 1: Fundamental algorithms* (3rd ed.). Addison-Wesley.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983). *Data structures and algorithms*. Addison-Wesley

Dasgupta, S., Papadimitriou, C. H., & Vazirani, U. V. (2008). *Algorithms*. McGraw-Hill.

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Java* (6th ed.). Wiley.

Brassard, G., & Bratley, P. (1996). *Fundamentals of algorithmics*. Prentice Hall.